

Problem Set 1
COMP301 Spring 2025
Week 1: 17.02.2025 - 23.02.2025

Instructions:

- This is a problem set for your own practice with the basics of scheme. It will not be graded.
- If you have not set up the DrRacket on your computer, you may download it at DrRacket.

Problem 1: In this problem, you will practice the elements of programming. Given the code block below, please find the results printed on the interpreter in response to each expression:

```
1) (+ 25 9 16)
2) (/ 24 4)
3) (+ (* 3 28) (- 2 2))
4) (define a 8)
5) (define b (+ a 7))
6) (+ a b (* a b))
7) (= a b)
8) (if (and (> b a) (< b (* a b)))
      b
      a)
9) (cond ((= a 9) 6)
        ((= b 3) (+ 6 7 a))
        (else 25))
10) (+ 10 (if (> b a) b a))
11) (* (cond ((> a b) a)
        ((< a b) b)
        (else -1))
    (+ a 15))
```

Problem 2: In this section, you will focus on procedures and recursion.

Part A. Given a list, implement a procedure named "idx_getter" that takes the given list and an integer n , and returns the n 'th element in that list (if the list is empty, then return an empty list). Check the *car* and *cdr* operations to see how to take subsets of the given list. What would we change if we would want to retrieve the sub-list of the given list between i 'th and j 'th index? You can test your implementation with the following code piece:

```
(idx_getter '(1 2 3 4 5 6) 0) ; returns 1
(idx_getter '(1 1 2 3 5 8 13 21) 4) ; returns 5
(idx_getter '() 0) ; returns '()
```

Part B. Implement a procedure, that takes a positive number n and returns the n 'th number of the sequence defined below. If n is given negative, then the procedure returns 0.

$$a_0 = 1$$
$$a_n = a_{n-1}^2 + 4$$

(a 1) $\rightarrow$ 5
(a 2) $\rightarrow$ 29
(a 3) $\rightarrow$ 845
(a 4) $\rightarrow$ 714029
(a 5) $\rightarrow$ 509837412845
(a 6) $\rightarrow$ 259934187536482970994029

Part C. Given a positive integer n , implement a procedure that returns $\#t$ if the number n is a prime number, otherwise it returns $\#f$. If n is given negative, then the procedure returns $\#f$.

Problem 1

1. 50
2. 6
3. 84
4. >a
8
5. >b
15
6. 143
7. #f
8. 15
9. 25
10. 25
11. 345

```
1 | #lang racket
2 | (displayln (+ 25 9 16))
3 | (displayln (/ 24 4))
4 | (displayln (+ (* 3 28) (- 2 2)))
5 | (define a 8)
6 | (define b (+ a 7))
7 | (displayln (+ a b (* a b)))
8 | (displayln (= a b))
9 | (displayln (if (and (> b a) (< b (* a b)))
10 | b
11 | a))
12 | (displayln (cond ((= a 9) 6)
13 | ((= b 3) (+ 6 7 a))
14 | (else 25)))
15 | (displayln (+ 10 (if (> b a) b a)))
16 | (displayln (* (cond ((> a b) a)
17 | ((< a b) b)
18 | (else -1))
19 | (+ a 15)))
20 |
```

Welcome to [DrRacket](#), version 8.15 [cs].

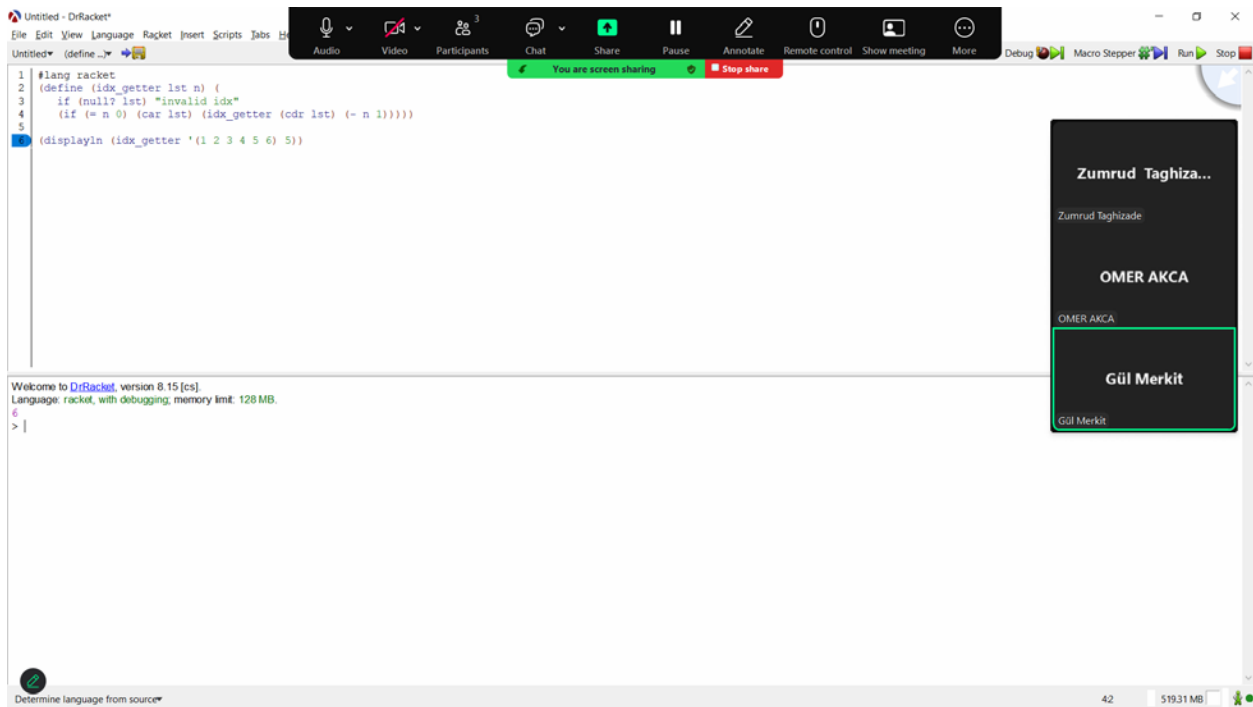
Language: **racket**, with debugging; memory limit: 128 MB.

```
50
6
84
143
#f
15
25
25
345
>
```

Problem 2

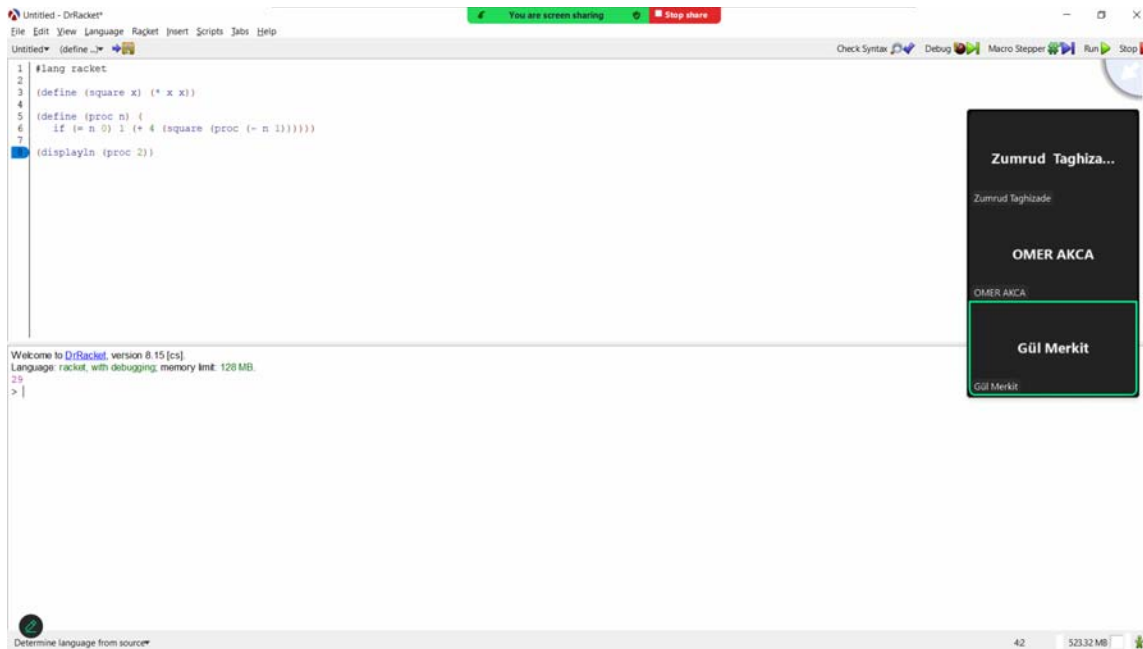
A.

```
(define (idx_getter lst n) (  
  if (null? lst) "invalid idx"  
      (if (= n 0) (car lst) (idx_getter (cdr lst) (- n 1)))))
```



B.

```
(define (square x) (* x x))  
(define (proc n) (  
  if (= n 0) 1 (+ 4 (square (proc (- n 1)))))
```



C.

```

(define (is_prime_helper n i) (if (< n (* i i)) #t (if (= n (* i i))
#f (if (= 0 (remainder n i)) #f (is_prime_helper n (+ 1 i))))))

```

```

(define (is_prime n) (
  if (<= n 0) #f
  (if (= n 1) #t
  (is_prime_helper n 2))))

```

Explanation: We check 1 separately as it is a special prime number. Then the recursion is entered, we are checking all numbers from 2 up to about the square root of the original number, because we know the numbers coming after the square root are impossible, as only their combinations with any of the checked numbers could possibly yield the number. When they are all eliminated, it is enough to stop at the square root. If it reaches the end, if it is not a perfect square, the program returns true. Else it returns false.

Untitled - DrRacket

File Edit View Language Racket Insert Scripts Tabs Help

Untitled (define...) Check Syntax Debug Macro Stepper Run Stop

```
1 #lang racket
2
3 (define (is_prime_helper n i) (if (< n (* i i)) #t (if (= n (* i i)) #f (if (= 0 (remainder n i)) #f (is_prime_helper n (+ 1 i))))))
4
5 (define (is_prime n) (
6   if (<= n 0) #f
7   (if (= n 1) #t
8     (is_prime_helper n 2))))
9
10 (displayln (is_prime 7))
11 (displayln (is_prime 70))
12 (displayln (is_prime 31))
```

Welcome to [DrRacket](#), version 8.15 [cs].
Language: racket, with debugging, memory limit: 128 MB.
#t
#f
#t
>

Determine language from source 20 530.12 MB